



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ**



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

**DNS TUNNELING SECURIZAT:
IMPLEMENTAREA UNUI PROTOCOL DE
TRANSPORT FIABIL ȘI REZILIENT LA RATE
LIMITING**

Absolvent

Stîngă Alexandru-Ionuț

Coordonator științific

Lect. dr. Bogdan Dumitru

București, iunie 2026

Rezumat

Concomitent cu progresul tehnologic, perimetrele de securitate ale sistemelor informatice și ale rețelelor au devenit mult mai stricte. Cu toate acestea, sistemul DNS încă rămâne un important vector de atac, susceptibil la comunicațiile ascunse. În această lucrare sunt prezentate structura, implementarea și evaluarea unui protocol securizat, bidirecțional, încadrat în tipologia Command-and-Control și exfiltrare de date. Acesta este construit exclusiv pe infrastructura DNS, având ca obiectiv principal conturarea unui nivel de transport fiabil, capabil să reziste mecanismelor de protecție automată a rețelelor, în special limitării ratei de transfer. Scopul este atins prin integrarea unui mecanism propriu de control al fluxului de trafic, de tip Stop-and-Wait ARQ. De asemenea, confidențialitatea și integritatea datelor sunt asigurate prin utilizarea algoritmilor ECDHE pentru stabilirea cheii de criptare, AES-GCM pentru criptarea și decriptarea autenticată a mesajelor, și HMAC cu un secret pre-partajat pentru prevenția atacurilor Man-in-the-Middle. Testele de performanță au demonstrat reziliența protocolului prin menținerea integrității datelor transferate în condiții de limitare a ratei impuse de ISP și de resolverii DNS. Așadar, lucrarea evidențiază amenințarea persistentă a canalelor ascunse și dovedește abilitatea primitivelor criptografice moderne, împreună cu un control riguros al fluxului, de a asigura transmiterea viabilă de date chiar și sub constrângerile dure impuse de rețelele contemporane.

Abstract

Alongside technological progress, the security perimeters of information systems and networks are hardening. Still, the Domain Name System remains an important attack vector, exploitable by covert communication. This thesis presents the structure, implementation and evaluation of a secure, bidirectional Command-and-Control and data exfiltration protocol. It is built exclusively over stateless DNS, its main objective being the engineering of a reliable transport layer capable of withstanding modern automated network defenses, specifically rate limiting. This is achieved by integrating a custom Stop-and-Wait Automatic Repeat Request mechanism. Furthermore, the confidentiality and integrity of the payload are enforced using Ephemeral Elliptic Curve Diffie-Hellman for the establishment of the cryptographic key, AES-GCM for authenticated message

encryption and decryption, and HMAC based on a master key for preventing Man-in-the-Middle attacks. The benchmarks have shown the resilience of the protocol by maintaining the integrity of the transferred data against the rate limiting of the ISP and DNS resolvers. Ultimately, the thesis emphasises the persistent threat brought by covert communication and proves the ability of modern cryptographic primitives, combined with strict flow control, to secure a viable data transfer channel even under contemporary network restrictions.

Cuprins

1	Introducere	6
1.1	Contextul actual	6
1.2	Obiectivele lucrării	6
1.3	Structura tezei	7
2	Preliminarii	8
2.1	Concepte fundamentale ale rețelei	8
2.1.1	Mecanismul de tunelare prin DNS	8
2.1.2	Limitările și protecțiile rețelelor	9
2.2	Primitive criptografice	9
2.2.1	Schimbul de chei (ECDHE)	10
2.2.2	Criptarea autenticată (AES-GCM)	10
2.2.3	Integritate și autentificare (HMAC)	11
2.3	Stadiul actual (State of the Art)	12
2.3.1	Iodine și DNSCat2	12
2.3.2	Sliver C2 și Noise Protocol Framework	13
3	Arhitectura și implementarea sistemului	13
3.1	Arhitectura generală	13
3.2	Structura pachetului	14
3.2.1	Antetul protocolului	14
3.2.2	Mecanismul de Flag-uri	15
3.2.3	Matematica fragmentării	15
3.3	Securitatea și managementul sesiunilor	16
3.3.1	Justificarea alegerilor criptografice	16
3.3.2	Fluxul de stabilire a sesiunii	17
3.3.3	Mecanisme defensive și izolare	17
3.4	Controlul fluxului (Stop-and-Wait ARQ)	18
3.4.1	Justificarea modelatorului de trafic implicit	18

3.4.2	Mecanismul de confirmare și retransmisie	18
3.4.3	Validarea integrității globale	20
3.5	Arhitectura Client-Server	20
3.5.1	Concurența serverului	20
3.5.2	Izolarea mediului, persistența și managementul memoriei	20
3.5.3	Fluxurile operaționale	21
4	Evaluarea Performanțelor și Rezultate	21
4.1	Metodologia de testare și mediul experimental	21
4.2	Analiza traficului la nivel de pachet	22
4.3	Analiza performanțelor și a rezilienței la rate limiting	23
5	Concluzii	24
5.1	Realizarea lucrării	24
5.2	Rezultate și aplicații practice	24
5.3	Direcții de dezvoltare ulterioară	25
Anexa 1	Parametrii curbei eliptice SECP256R1	26
Anexa 2	Arhitectura operațională AES-GCM	26
Anexa 3	Analiza detaliată a pachetelor în Wireshark	28
	Referințe	30

1 Introducere

1.1 Contextul actual

În contextul curent al securității cibernetice, zonele de protecție ale rețelelor private au devenit foarte restrictive. Folosirea pe scară extinsă a aplicațiilor avansate de tip Firewall, precum NGFW, și a sistemelor de detectare și protecție împotriva infiltrărilor neautorizate, asemenea IDS sau IPS, a dus la blocarea automată a majorității porturilor și protocoalelor de comunicație ce nu sunt esențiale. Cu toate acestea, există o excepție arhitecturală esențială, anume Domain Name System (DNS).

Întrucât rezoluția numelor de domeniu reprezintă o componentă importantă pentru funcționarea internetului, traficul pe portul 53, adesea folosit de protocoalele UDP și TCP, este rar filtrat sau blocat în întregime. Din acest motiv, infrastructura DNS devine un vector de atac propice stabilirii canalelor ascunse. Astfel, prin încapsularea datelor arbitrare în interiorul cererilor și răspunsurilor DNS legitime, tehnică cunoscută sub numele de DNS Tunneling, comunicarea reușește să ocolească politicile stricte de securitate impuse de sistemele informatice. Deși inițial utilizată pentru evitarea portalurilor captive din cadrul rețelelor publice, tehnica a fost răspândită în cadrul exploatărilor de tip Command-and-Control și pentru exfiltrarea silențioasă a datelor din medii izolate.

1.2 Obiectivele lucrării

Principalul scop al acestei lucrări de licență, încadrată în subdomeniul securității cibernetice și al rețelelor de calculatoare, este planificarea structurii, implementarea și evaluarea unui protocol de transport personalizat. Acesta garantează o comunicare bidirecțională fiabilă și sigură doar prin utilizarea QNAME-urilor și a înregistrărilor de tip TXT din DNS.

Spre deosebire de aplicațiile tradiționale ce au la bază mecanisme de tip *Sliding Window*, lucrarea prezintă o arhitectură adaptată să reziste la mecanismele moderne de apărare, mai ales la limitarea ratei de transfer dictată de furnizorii de servicii internet și resolverii publici. Obiectivele concrete includ:

- **Dezvoltarea unui nivel de transport fiabil:** Proiectarea unui sistem de control al fluxului

de tip Stop-and-Wait Automatic Repeat Request, care să gestioneze pierderile de pachete (de exemplu, *ACK loss*) și să funcționeze pe post de formator implicit de trafic pentru evitarea declanșării protecțiilor anti-DDoS.

- **Securizarea canalului de comunicație:** Implementarea unui sistem criptografic solid ce respectă triada CIA: Confidențialitate, Integritate și Disponibilitate. Se utilizează schimbul de chei Ephemeral Elliptic Curve Diffie-Hellman pentru obținerea cheii de criptare simetrică, AES-GCM pentru criptarea autenticată a datelor și HMAC bazat pe un secret pre-partajat pentru prevenirea atacurilor de tip Man-in-the-Middle.
- **Implementarea funcționalităților de Command-and-Control:** Crearea unei arhitecturi de tip client-server multi-thread capabilă să susțină încărcarea și descărcarea fișierelor, dar și execuția într-un mediu securizat a comenzilor shell la distanță.

1.3 Structura tezei

Structura lucrării este:

Capitolul 2: Preliminarii prezintă baza teoretică pe care protocolul este construit, extrapolând funcționarea canalelor ascunse prin DNS, constrângerile inerente acestora, dar și primitivele criptografice moderne utilizate. De asemenea, acest capitol include o analiză a soluțiilor actuale, evidențiindu-se necesitatea abordării propuse.

Capitolul 3: Arhitectura și implementarea sistemului detaliază principalul grad de inovație realizat în lucrare. Sunt reliefate designul modular al aplicației, matematica fragmentării datelor pentru trimiterea pachetelor, mașina de stări a procesului de *handshake*, cât și logica mecanismului de Stop-and-Wait ARQ pentru tratarea erorilor ce pot apărea pe rețea.

Capitolul 4: Evaluarea Performanțelor și Rezultate descrie metoda de testare și analizează performanța protocolului într-un context real de rețea. Se interpretează rezultatele referitoare la asimetria lățimii de bandă și latenței, demonstrându-se totodată reziliența arhitecturii la limitările ratei de transfer. De asemenea, se realizează o analiză a pachetelor folosind *Wireshark*.

Capitolul 5: Concluzii rezumă rezultatele obținute, validează îndeplinirea obiectivelor propuse și discută posibilele direcții de dezvoltare și optimizarea ulterioară a protocolului.

2 Preliminarii

Acest capitol stabilește fundamentul teoretic necesar pentru a înțelege arhitectura protocolului personalizat. În continuare, se explică funcționarea sistemului DNS ca mediu de transport și se analizează setul de primitive criptografice alese pentru securizarea canalului de comunicare. De asemenea, tot în această secțiune este prezentat stadiul curent al cercetării în domeniul canalelor ascunse, împreună cu limitările inerente ale soluțiilor existente.

2.1 Concepte fundamentale ale rețelei

Analiza modului de operare al sistemului DNS este esențială. Prin natura sa, DNS este un protocol *stateless* bazat pe UDP (portul 53), conceput exclusiv pentru rezoluția numelor de domeniu. Serverul tratează fiecare cerere independent, clientul fiind obligat să trimită constant toate datele de stare.

2.1.1 Mecanismul de tunelare prin DNS

Deși rolul DNS-ului este corespondența nume-IP, neglijarea securității acestui protocol permite exploatarea precum *DNS Tunneling*. Arhitectura cere un domeniu controlat, un client compromis și un server de tunelare configurat ca *name server autoritativ* pentru acel domeniu [1].

Deoarece informațiile exfiltrate nu pot fi transmise în text clar, ele sunt codificate. Metodele comune includ *Base32* pentru atașarea datelor ca subdomeniu în eticheta QNAME și *Base64* pentru secțiunea RDATA [1].

Standardul pentru o tunelare robustă folosește înregistrări **TXT**, care permit șiruri arbitrare de până la 255 de octeți. Răspunsurile DNS pot conține multiple șiruri TXT, permițând un transfer *downstream* superior celui *upstream*. Mai mult, extensia EDNS depășește limita tradițională de 512 octeți a pachetelor UDP. Totuși, protocolul impune restricții stricte QNAME-urilor: o etichetă (porțiunea delimitată de puncte) nu poate depăși 63 de octeți, iar lungimea totală a numelui de domeniu este limitată la 255 de octeți [1].

2.1.2 Limitările și protecțiile rețelelor

Rețelele sunt prudente cu protocoalele UDP deoarece facilitează atacurile de tip *Denial-of-Service* (DoS). DNS este o țintă perfectă din trei motive principale [2]:

- **Lipsa verificării sursei:** Software-ul (ex. BIND) nu poate valida sursa pachetului, permițând falsificarea adresei pentru a viza o victimă.
- **Politici ISP permissive:** Lipsa filtrării pachetelor falsificate la ieșirea din rețea facilitează atacurile de reflectare (*Reflection Attacks*).
- **Asimetria traficului:** O interogare ANY de 36 de octeți poate genera răspunsuri de peste 3500 de octeți. Această amplificare de 100 de ori este fundația atacurilor *DNS Amplification*.

Deoarece filtrarea răspunsurilor legitime este dificilă, operatorii utilizează *Response Rate Limiting* (RRL). Acest mecanism detectează răspunsurile similare frecvente folosind o schemă *token bucket*. Fiecare combinație client-răspuns consumă un credit; la epuizarea acestuia, pachetele sunt aruncate (*packet loss*), limitând rata de transfer [2].

Suplimentar, echipamentele *Next-Generation Firewalls* (NGFW) utilizează analize statistice. Deoarece *payload*-ul este codificat în QNAME, domeniile rezultate prezintă o entropie informațională ridicată față de cele standard. Tunelele maximizează transferul (etichete de 63 de octeți, domenii de 255 de octeți) și generează nume cu o densitate nefirească de cifre. Din acest motiv, tunelurile tradiționale sunt foarte susceptibile la detecția automată [1].

2.2 Primitive criptografice

Securitatea oferită de protocoalele criptografice tradiționale (precum TLS) este dificil de implementat peste DNS din cauza constrângerilor de dimensiune a pachetelor și a naturii *stateless* specifice UDP. Așadar, lucrarea utilizează o suită de primitive moderne, selectate pentru eficiența spațială și robustețea matematică în context pre-cuantic.

2.2.1 Schimbul de chei: Ephemeral Elliptic Curve Diffie-Hellman (ECDHE)

Diffie-Hellman (DH) stabilește un secret comun pe un canal nesecurizat. Algoritmul implică un grup G de ordin p (număr prim mare) cu un generator g . Părțile generează $a, b \in \mathbb{Z}_p^x$ și schimbă cheile publice $h_A = g^a$ și $h_B = g^b$, calculând secretul comun $K_A = h_B^a = h_A^b = K_B$. Securitatea se bazează pe dificultatea calculării logaritmului discret. Criptografia pe curbe eliptice (ECC) aplică aceste principii pe structura algebrică a curbilor eliptice, oferind un nivel de securitate echivalent dar utilizând chei semnificativ mai mici (*Elliptic Curve Diffie-Hellman* - ECDH).

Modul efemer (ECDHE) utilizează cheile generate pentru o singură sesiune. Odată stabilit secretul comun, calculele anterioare sunt eliminate, garantând *Perfect Forward Secrecy* (PFS); compromiterea ulterioară a unei chei nu permite decriptarea traficului trecut. Conform standardelor NIST [3], secretul comun nu trebuie utilizat direct ca o cheie de criptare simetrică sau ca vector de inițializare, ci trebuie procesat printr-o schemă de derivare, precum *HKDF* [4], care execută doi pași:

- $\text{HKDF-Extract}(\text{salt}, \text{IKM}) \rightarrow \text{PRK}$
- $\text{HKDF-Expand}(\text{PRK}, \text{info}, L) \rightarrow \text{OKM}$
 - **salt** - valoare aleatoare opțională (implicit, un șir de zerouri de lungime *HashLen*).
 - **IKM** - materialul inițial (*Input Keying Material* - secretul comun ECDHE).
 - **PRK** - cheie pseudoaleatoare de lungime *HashLen*.
 - **info** - context opțional pentru legarea cheii de aplicație.
 - **L** - lungimea țintă a cheii.
 - **OKM** - cheia derivată de ieșire.

2.2.2 Criptarea autenticată: Advanced Encryption Standard in Galois/Counter Mode (AES-GCM)

AES este un algoritm de criptare pe blocuri de 128 biți, cu lungimi ale cheii de 128, 192 sau 256 biți. Independent, AES asigură doar confidențialitatea.

Galois/Counter Mode (GCM) este un mod de operare care extinde AES (modul *Counter*) pentru a oferi criptare autenticată. Autenticitatea se obține printr-o funcție hash (*GHASH*) pe

un câmp binar Galois, detectând orice modificare malițioasă sau accidentală [5]. Detaliat în Anexa 2. GCM criptează datele și calculează o etichetă (*tag*) pentru întregul mesaj, definind funcțiile:

$$\text{GCM-AE}_K(IV, P, A) = (C, T)$$

$$\text{GCM-AD}_K(IV, C, A, T) = P \text{ or } FAIL$$

unde:

- **K** - cheia de criptare.
- **IV** - vectorul de inițializare (*nonce*).
- **P** - textul în clar.
- **C** - textul cifrat.
- **A** - date adiționale autentificate.
- **T** - eticheta de autentificare (*tag*).

Pentru a menține securitatea funcției hash, probabilitatea reutilizării perechii (IV, K) trebuie să fie sub 2^{-32} .

2.2.3 Integritate și autentificare: Hash-based Message Authentication Code (HMAC)

Un *Message Authentication Code* (MAC) validează informația între entități care împart un secret. Varianta *HMAC* folosește funcții hash criptografice iterative, securitatea sa depinzând de rezistența funcției utilizate [6]. Calculul HMAC este:

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$$

unde:

- **H** - funcția hash.
- **B** - dimensiunea blocului (în octeți).
- **L** - dimensiunea ieșirii funcției *H*.
- **m** - mesajul autentificat.
- **len(x)** - lungimea în biți a valorii *x*.

- **K** - cheia secretă ($\text{len}(K) \geq L$; dacă $\text{len}(K) > B$, se folosește $H(K)$).
- **ipad** - octetul 0x36 repetat de B ori.
- **opad** - octetul 0x5C repetat de B ori.

Pentru a spori securitatea și a limita expunerea informațiilor în cazul unei criptanalize, ieșirea poate fi trunchiată păstrând doar cei mai din stânga t biți ($t \geq L/2$).

2.3 Stadiul actual (State of the Art)

Exploatarea vulnerabilităților prezente în DNS pentru exfiltrarea datelor, ocolirea Firewall-urilor și menținerea infrastructurilor de tip Command-and-Control a fost un subiect profund documentat în literatura de specialitate. Cu toate acestea, majoritatea soluțiilor existente nu reușesc să găsească un compromis între funcționalitatea brută și securitatea operațională, împreună cu fiabilitatea. Acestea decid să prioritizeze una dintre ele în detrimentul celeilalte.

2.3.1 Iodine și DNSCat2

Iodine și DNSCat2 sunt instrumente clasice de tunelare, axate pe funcționalitate pură, dar au lacune la nivel de securitate operațională și eficiență pe rețea.

Iodine [7] este conceput să creeze un tunel IPv4, utilizând interfețele TUN/TAP, peste DNS. Acesta prezintă o utilitate cuprinzătoare la nivelul rețelei, însă înglobarea pachetelor IP (cu antete mari) în înregistrări DNS conduce la o fragmentare majoră. Deci, se generează un volum masiv de cereri, lucru ce face Iodine să fie susceptibil la detectarea prin analiză volumetrică și la limitarea ratei de transfer. Pe lângă faptul că este „zgomotos”, instrumentul nu dispune nativ de o suită de primitive criptografice moderne. Utilizatorul este obligat să tuneleze protocoale complementare, precum SSH, pentru obținerea confidențialității, deci se creează mai mult trafic redundant.

DNSCat2 [8] este eficientizat strict pentru stabilirea canalelor de *Command-and-Control* (C2). Suportă criptarea traficului, dar arhitectura criptografică folosită este arhaică comparativ cu standardele actuale. Protocolul nu asigură garanții prin *Perfect Forward Secrecy* (PFS) sau prin criptare autenticată standardizată (AEAD). Este, astfel, nepotrivit și fragil în raport cu sistemele contemporane de analiză a traficului.

2.3.2 Sliver C2 și Noise Protocol Framework

La cealaltă extremă din perspectiva securității se află **Sliver C2** [9], un sistem avansat de securitate ofensivă ce prioritizează evaziunea și criptografia superioară. Pentru a securiza traficul, acesta include *Noise Protocol Framework* [10], un standard criptografic ce asigură confidențialitatea, autentificarea și PFS, folosind primitive precum Curve25519, AES-GCM, ChaCha20.

Totuși, nivelul de securitate ridicat periclitează funcționalitatea fiabilă peste DNS. *Handshake*-ul din *Noise* a fost proiectat pentru medii TCP sigure, deci, pe DNS realizează numeroase schimburi de pachete criptografice mari. Luând în considerare instabilitatea UDP, cu multe pierderi de pachete și filtre RRL, traficul din *handshake* este fragmentat și stabilirea sesiunii eșuează. Deși este excelent criptografic, nu este un tunel DNS rezistent din cauza lipsei unui sistem de controlare a fluxului conceput pentru acest protocol.

3 Arhitectura și implementarea sistemului

Capitolul conturează arhitectura tehnică a protocolului. Pentru a echilibra compromisurile observate în instrumentele existente, acesta elimină necesitatea utilizării standardelor de nivel înalt, precum TCP over DNS. Sistemul este proiectat să includă securitatea nativ, combinând criptografia pe curbe eliptice și cea autentificată cu un mecanism strict de control al fluxului. Astfel, controlul congestiei este transformat într-un modelator de trafic ce îngreunează detecția volumetrică și asigură rezistența sesiunii în fața pierderii pachetelor cauzate de procesul de *Rate Limiting* impus de resolverii publici.

3.1 Arhitectura generală

Sistemul operează asincron, urmând modelul de rețea de tip client-server, și încapsulează traficul bidirecțional doar în înregistrări DNS TXT. Modularizarea arhitecturii facilitează testarea, securitatea și mentenanța, respectându-se principiul separării responsabilităților (Tabelul 3.1).

Modul	Responsabilitate Principală
<code>server.py</code>	Gestionează <i>socket</i> -ul UDP (portul 53), multiplexează sesiunile și rotează pachetele decriptate către <i>thread pool</i> .
<code>client.py</code>	Generează interogările DNS (Scapy), menține starea locală a sesiunii și execută sarcinile (Upload/Download/Comenzi).
<code>crypto_utils</code>	Încapsulează operațiunile criptografice: generarea cheilor ECDHE, derivarea HKDF, criptarea AES-GCM și validarea HMAC.
<code>transport.py</code>	Oferă clasa <code>Packet</code> pentru serializarea/deserializarea binară a antetelor și gestionează fragmentarea fișierelor.
<code>worker_pool</code>	Execută sarcinile concurrent folosind redimensionare dinamică (scade/crește numărul de procese), prevenind blocarea serverului.

Tabela 3.1: Structura modulară a sistemului de tunelare

3.2 Structura pachetului

Întrucât UDP este inerent *stateless* și există limite stricte de dimensiune dictate de DNS, a fost implementat un protocol la nivel de aplicație pentru realizarea comunicării. Orice mesaj transmis este serializat într-un format compus dintr-un antet de 9 bytes și un *payload*.

3.2.1 Antetul protocolului

Antetul conține doar elementele esențiale pentru multiplexarea sesiunilor și controlarea fluxului. Structura pachetului este prezentată în Figura 3.1.

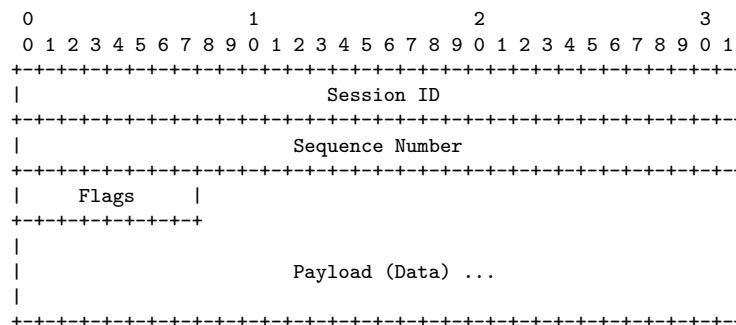


Figura 3.1: Diagrama structurii antetului de pachet

- **Session ID (32 biți):** Identificator unic al sesiunii; Necesari în obținerea *IV*-ului din criptarea *AES-GCM*
- **Sequence Number (32 biți):** Contorizează ordinea pachetelor (*seq_num* la trimitere, *ack_num* la recepție). Garantează unicitatea *IV*-ului pentru fiecare criptare.
- **Flags (8 biți):** Octet de stare pentru rutarea, tipul și operația pachetului.

3.2.2 Mecanismul de Flag-uri

Controlul logic al sesiunii este gestionat de octetul de flag-uri. Clasa Packet folosește operatori la nivel de bit (*bitwise*) pentru a combina multiple stări. Flag-urile sunt detaliate în Tabelul 3.2.

Bit	Hexazecimal	Flag	Descriere / Rol
0	0x01	SYN	Inițializarea sesiunii prin <i>handshake</i> .
1	0x02	ACK	Confirmarea recepției (<i>Acknowledge</i>).
2	0x04	FIN	Marcarea ultimului <i>chunk</i> (final transmisie).
3	0x08	DAT	Arată prezența datelor în <i>payload</i> .
4	0x10	UPL	Sesiune de tip <i>Upload</i> (Client → Server).
5	0x20	DWN	Sesiune de tip <i>Download</i> (Server → Client).
6	0x40	CMD	Sesiune de execuție comenzi (<i>Command Execution</i>).
7	0x80	RPC	Continuarea execuției unei comenzi precedente (<i>Rest of Previous Command</i>).

Tabela 3.2: Tabelul alocării flag-urilor pe 8 biți

3.2.3 Matematica fragmentării

Înainte de transmitere, pachetul serializat este criptat cu AES-GCM și i se adaugă o etichetă de 16 bytes. DNS impune limite stricte, deci dimensiunea datelor din *payload* trebuie calculată cu rigoare în baza asimetriei codificărilor.

Traficul Upstream (Codificare Base32) Cererile către server sunt înglobate în QNAME. DNS restricționează lungimea unui domeniu la 255 de octeți, iar a unei etichete la cel mult 63 de octeți. Știind $L_{criptat}$ lungimea mesajului cifrat, dimensiunea textului codificat Base32 este:

$$\text{Dimensiune}_{\text{Base32}} = \left\lceil \frac{L_{criptat} \times 8}{5} \right\rceil \text{ octeți}$$

Astfel, variabila UPSTREAM_SIZE a fost fixată la 100 bytes, fiindcă șirul final, concatenat cu domeniul de bază și limitatorii ". ", nu trebuie să depășească 255 bytes.

Trafic Downstream (Codificare Base64) Răspunsurile dinspre server folosesc înregistrări TXT, în care un string TXT (de maxim 255 bytes) utilizează codare Base64. Astfel, dimensiunea

textului codificat este:

$$\text{Dimensiune}_{\text{Base64}} = \left\lceil \frac{L_{\text{criptat}} \times 4}{3} \right\rceil \text{ octeți}$$

Mai mult, DNS permite trimiterea unor liste de stringuri TXT într-o înregistrare TXT. Deci, variabila `DOWNSTREAM_SIZE` este fixată la 800 de octeți, datorită extensiei EDNS ce permite trimiterea mai multor date, dar și pentru respectarea restricției de 1232 bytes impusă de DNS Flag Day 2020 (dimensiunea pachetului final de ≈ 1100 bytes).

3.3 Securitatea și managementul sesiunilor

Implementarea fazei de *handshake* și managementul sesiunilor sunt gestionate de principiul apărării în adâncime, ajustat la limitările dimensionale ale DNS-ului.

3.3.1 Justificarea alegerilor criptografice

Schimbul de chei asimetric brut (precum ECDH) oferă confidențialitate, dar nu și integritate. Nu posedă mecanisme de autentificare a identității, deci este susceptibil la atacuri Man-in-the-Middle (MitM). Soluțiile clasice rezolvă problema utilizând semnături digitale asimetrice (RSA, ECDSA), abordarea fiind nepotrivită în contextul tunelării DNS. Semnătura RSA-2048 are 256 bytes, deci includerea semnăturii publice creează fragmentare mare din cauza depășirii limitei dimensionale a cererilor DNS.

Din cauza restricțiilor mediului, s-au luat următoarele decizii de proiectare:

- **ECDHE în detrimentul RSA:** S-a ales *ECDHE* (pe curba eliptică SECP256R1) deoarece dispune de chei publice mici (pe 64 de octeți) și garantează *PFS*. Parametrii matematici ai curbei SECP256R1 sunt detaliați în Anexa 1. Compromiterea cheii serverului nu permite decriptarea sesiunilor trecute, spre deosebire de transportul cheilor RSA.
- **HMAC-SHA256 cu PSK în detrimentul Semnăturilor Digitale:** Pentru autentificarea mutuală se folosește un secret pre-partajat (*PSK*) verificat prin HMAC. Mecanismul oferă integritate și autentificare simetrică, necesitând un spațiu suplimentar de 32 bytes, potrivit pentru constrângerile QNAME-ului.

3.3.2 Fluxul de stabilire a sesiunii

Canalul sigur este stabilit printr-un *handshake* în trei pași, integrat în *server.py* și *client.py*.

1. **Cererea de Sincronizare (Client → Server):** Clientul generează un *Session ID* întreg aleatoriu și o pereche de chei ECDHE. Cheia publică este împachetată împreună cu flag-ul SYN. Blocul este semnat cu *HMAC-SHA256* utilizându-se PSK-ul local. Semnătura se concatenează payload-ului. Pachetul codificat în Base32 este trimis ca subdomeniu DNS.
2. **Validarea și Răspunsul (Server → Client):** Serverul primește cererea și recalculează HMAC-ul folosind propriul PSK, iar dacă se potrivește, verifică unicitatea *Session ID*-ului, prevenind atacurile de *Session Fixation*. Apoi, serverul își generează cheile efemere și calculează secretul comun. Obține cheia finală AES-GCM (pe 256 biți) procesând secretul ECDH prin HKDF. Acesta trimite cheia sa publică cu flag-urile SYN | ACK, protejate de un nou HMAC.
3. **Confirmarea (Client):** Clientul primește răspunsul, confirmă identitatea serverului și își calculează cheia simetrică. Apoi, sesiunea trece în modul ESTABLISHED.

3.3.3 Mecanisme defensive și izolare

În faza de *handshake*, procesarea restrictivă a pachetelor reprezintă baza apărării în adâncime. Dacă un pachet SYN are HMAC invalid sau un format greșit, serverul întoarce un răspuns TXT gol. Beneficiile acestei abordări sunt:

- **Obfuscarea amprente:** Fiindcă se returnează răspunsuri goale, serverul se comportă ca un DNS resolver legitim ce nu are înregistrările domeniului. Semnătura instrumentului de C2 rămâne neobservabilă la scanările active de rețea.
- **Anularea factorului de amplificare:** Atacurile de *DNS Amplification* se bazează pe servere care generează răspunsuri mai mari decât cererile primite. Deoarece cererile neautorizate primesc răspuns gol, serverul nu poate fi utilizat în atacuri *DDoS*.
- **Gestiunea duplicatelor UDP:** Fiindcă UDP este instabil, pachetele SYN pot ajunge duplicate la destinație. Dacă identifică o reinițializare a unei sesiuni active, nu regenerează

cheile ECDHE, întoarce răspunsul din cache, prevenindu-se coruperea stării criptografice.

La finalizarea handshake-ului, cheile ECDHE sunt șterse, iar pachetele ulterioare sunt multiplexate doar pe baza `Session ID`-ului, fiind protejate de criptarea autenticată AES-GCM.

3.4 Controlul fluxului (Stop-and-Wait ARQ)

Deoarece UDP peste DNS este stateless, nu garantează livrarea pachetelor și este susceptibil la pierderea lor, protocolul implementează un sistem de control al fluxului. Acesta este bazat pe *Stop-and-Wait ARQ*, complementat de validarea integrității prin SHA-256.

3.4.1 Justificarea modelatorului de trafic implicit

Protocoalele clasice de transfer utilizează *Sliding Windows* pentru a trimite pachete în *burst*-uri. Această abordare nu este potrivită în cazul tunelării DNS, fiindcă trimiterea concurentă a mai multor cereri către un resolver activează *RRL-ul*, deci IP-ul sursă se blochează.

Stop-and-Wait ARQ permite inherent existența unui singur pachet neconfirmat în rețea. Ritmul de transmisie depinde de timpul de răspuns al rețelei (*Round-Trip Time* - RTT). Acest comportament reduce *spike*-urile de interogări și acționează ca un modelator de trafic, frecvența fiind ținută sub nivelul de detecție al resolverelor.

3.4.2 Mecanismul de confirmare și retransmisie

Circuitul unui transfer de date este controlat de metoda `send_package`, ce gestionează retransmisiile și verifică starea canalului conform regulilor protocolului:

Algoritmul 3.1 Logica Stop-and-Wait ARQ (pentru un chunk)

Require: `seq_num` (numărul secvenței), `flags`, `expected_flag`, `data`

- 1: `packet` $\leftarrow$ CrijteazăȘiCodifică(`session_id`, `seq_num`, `flags`, `data`)
- 2: `qname` $\leftarrow$ CreazăInterogareDNS(`packet`)
- 3: **for** `attempt` = 1 până la `max_retries` **do**
- 4: `response` $\leftarrow$ TrimiteCererea(`qname`, timeout = 3s)
- 5: **if** `response` este Null **then**
- 6: **continue**

▷ Timeout atins

```

7:   end if
8:   dec_resp ← DecripteazăPachet(response)
9:   if decriptarea eșuează then
10:      continue                                ▷ Integritate compromisă
11:   end if
12:   if dec_resp.ack_num == seq_num și dec_resp.flags conține expected_flag then
13:      return dec_resp                          ▷ Pachet conform formatului
14:   end if
15: end for
16: return Eroare                                ▷ Transfer eșuat

```

1. **Transmiterea și Blocarea:** Pentru fiecare *chunk* citit de Fragmenter, clientul transmite pachetul format cu *seq_num*. Acesta așteaptă răspunsul, cu un *timeout* de 3s pe interogare.

2. **Validarea în Client:** La decriptarea răspunsului, pachetul este acceptat doar dacă *ack_num* = *seq_num* și conține flag-ul de confirmare (ACK sau FIN) și cel al operației așteptate (*expected_resp_flag*). Altfel, protocolul se consideră încălcat.

3. **Validarea în Server:** Pentru a opri atacurile de tip *Replay* sau *Reentry*, serverul reține secvența așteptată (*last_written_seq* + 1). La primirea unui pachet de date:

- *seq_num* == *expected_seq*: Datele sunt scrise, iar starea avansează.
- *seq_num* < *expected_seq*: Pachet duplicat (ACK rătăcit pe rețea). Datele sunt ignorate pentru a preveni coruperea fișierului, dar serverul retrimite un ACK.
- *seq_num* > *expected_seq*: Pachet viitor ce indică o eroare. Pachetul este aruncat.

4. **Retransmisia și limitarea erorilor:**

- Dacă clientul primește confirmarea corectă, trece la următorul *chunk*.
- Dacă *timeout*-ul expiră, se primește o secvență greșită sau lipsesc flag-urile așteptate, se consideră ca fiind o pierdere. Pachetul se retransmite automat.
- Pentru a nu se rula indefinit, protocolul dictează o limită de 6 retransmiteri. Depășirea acestei restricții declanșează o eroare *FATAL*, iar transferul este anulat.

3.4.3 Validarea integrității globale

Pentru prevenirea coruperii datelor în timpul asamblării, înainte de începerea transferului se trimite un pachet de metadate, ce conține numărul total de fragmente și hash-ul SHA-256 al fișierului inițial.

La finalul transferului, solicitantul calculează hash-ul fișierului asamblat și îl compară cu cel primit. Doar după validarea integrității se transmite pachetul FIN pentru încheierea sesiunii.

3.5 Arhitectura Client-Server

Sistemul este gândit să mențină execuția în paralel și să izoleze activitățile de rețea de cele de procesare a datelor. Logica operațională este repartizată între client și serverul de tunelare.

3.5.1 Concurența serverului

Pentru a împiedica *bottleneck*-urile pe socket-ul UDP central, server .py funcționează ca un *dispatcher*. După interceptarea unui pachet valid, prelucrarea acestuia se delegă lui `WorkerPool`.

Se utilizează un sistem de *Thread Pool* cu redimensionare dinamică, extensia fiind făcută imediat, pe când micșorarea după un *buffer* de inactivitate de 30s. Starea cozii de așteptare declanșează scalarea activă, thread-ul principal rămânând liber pentru recepția pachetelor.

3.5.2 Izolarea mediului, persistența și managementul memoriei

Reziliența, portabilitatea și securitatea infrastructurii sunt asigurate prin containerizarea serverului în Docker. Mecanismele interne de gestionare a stării sunt:

- **Sandboxing și Recuperare Automată:** Serverul rulează într-un mediu izolat de sistemul de operare gazdă, atenuându-se riscul de compromitere al infrastructurii. Prin politica `restart: unless-stopped` din orchestrator, serverul își revine automat la apariția oricărei erori majore.
- **Persistența sesiunilor:** Starea sesiunilor active este salvată în `sessions.json`. Prin *Volume Binding*, fișierul de stare și directorul `uploads/` persistă pe gazdă. Când containerul este repornit, transferurile întrerupte pot continua datorită restaurării sesiunilor.

- **Garbage Collector (GC):** O procedură rulează recurent pentru a găsi sesiunile inactive de mai mult de 300s. GC-ul forțează închiderea *file handle*-urilor și ștergerea fișierelor temporare ce aparțin sesiunilor moarte, prevenind *Resource Exhaustion*.

3.5.3 Fluxurile operaționale

Clientul controlează operarea serverului prin flag-urile de protocol. Există trei funcții:

- **Upload - UPL:** Procesul este inițiat de client prin trimiterea unui pachet cu metadate. După *Acknowledgement*, clientul transmite fragmentele prin *Stop-and-Wait ARQ*. La final, serverul validează integritatea fișierului asamblat și îl salvează.
- **Download - DWN:** Clientul solicită un fișier de pe server. Dacă fișierul există, serverul răspunde cu metadatele corespunzătoare. Apoi, clientul face cereri DNS succesive pentru fiecare fragment, reconstruiește fișierul local și îi validează integritatea la final.
- **Remote Command Execution - CMD:** Comenzile destinate serverului sunt filtrate, cele malițioase (precum `rm -rf /`) fiind blocate. Cele permise sunt executate de server într-un sub-proces (`subprocess.Popen`) cu un *timeout* de 1.5s pentru prevenția blocajelor. Ieșirea standard este salvată într-un fișier temporar, iar clientul se mută pe operația de DWN a rezultatului. La final, serverul declanșează *cleanup*-ul fișierului temporar.

4 Evaluarea Performanțelor și Rezultate

Rezultatele obținute în urma testării protocolului sunt prezentate în acest capitol. În analiză sunt evaluate eficiența transferului de date, capacitatea de evaziune și reziliența sistemului de control al fluxului în situații de rețea reale.

4.1 Metodologia de testare și mediul experimental

În testele de performanță s-au utilizat un client local și un server găzduit pe un Virtual Private Server (VPS) de la *Digital Ocean*. Pentru testarea comparativă s-a folosit Docker Compose (prin

comanda `docker compose up -d --build`), ce a instanțiat concomitent serverul DNS pentru tunelare (UDP, port 53) și un server TCP (port 9000) aplicat ca *baseline*.

Pentru menținerea veridicității experimentelor, datele de test au avut entropie maximă, fiind generate pseudoaleator (`os.urandom`). Astfel, sunt prevenite optimizările de compresie la nivel de rețea sau protocol. Dimensiunea fișierelor variază progresiv de la 5 KB la 1 MB. Performanța protocolului personalizat a fost comparată cu *baseline*-ul TCP, ce are eficiență maximă teoretică la transferul pe canalul de comunicație.

4.2 Analiza traficului la nivel de pachet

S-a utilizat utilitarul Wireshark [11] prin care traficul a fost capturat și analizat pentru a dovedi caracterul evaziv al comunicării. Întreaga comunicație este compusă din cereri și răspunsuri DNS standard de tip TXT. Datele sunt ofuscate prin codificarea Base32/Base64 și securizate prin criptarea AES-GCM.

Pachetele în trafic *upstream* ascund instrucțiunile directe în interiorul câmpului QNAME, pe când în trafic *downstream* serverul exploatează extensia EDNS pentru a trimite răspunsuri TXT de peste 1200 bytes. Capturile de ecran aferente pot fi consultate în **Anexa 3**.

Conținutul comunicației rămâne neclar în lipsa PSK-ului. Deși entropia ridicată, folosirea înregistrărilor TXT și dimensiunea nefirească a răspunsurilor pot fi marcate ca anomalii de către instrumentele de analiză euristică și statistică, *firewall*-urile DPI, ce verifică doar formatul protocolului, vor permite tranzitul. Structural, pachetele sunt conforme standardelor RFC, pe când criptarea AES-GCM asigură imposibilitatea decodificării neautorizate sau a interceptării active.

Mai mult, Figura 4.1 confirmă ritmul de transmisie dictat de *Stop-and-Wait ARQ*. Se observă alternanța dintre interogările scurte ale clientului și răspunsurile mari ale serverului (trafic *downstream*), confirmând absența *burst*-urilor de trafic.

27638	2023.7985726...	192.168.1.4	8.8.8.8	DNS	150 Standard query 0xea6c TXT LB2LQDH5LM4YS6Q6LV3LCF0ZCTNTERAAAF2LMH.30.4006359817.server.ownprotocol.ninja OPT
27639	2023.8462586...	8.8.8.8	192.168.1.4	DNS	1267 Standard query response 0xea6c TXT LB2LQDH5LM4YS6Q6LV3LCF0ZCTNTERAAAF2LMH.30.4006359817.server.ownprotocol.ninja TXT OPT
27640	2023.9765989...	192.168.1.4	8.8.8.8	DNS	150 Standard query 0x12bc TXT RVVT7J77SZAMQ7JCONGZ038CZBA6TXNI773YSC5.31.4006359817.server.ownprotocol.ninja OPT
27641	2023.9323926...	8.8.8.8	192.168.1.4	DNS	1267 Standard query response 0x12bc TXT RVVT7J77SZAMQ7JCONGZ038CZBA6TXNI773YSC5.31.4006359817.server.ownprotocol.ninja TXT OPT

Figura 4.1: Captură Wireshark: Alternanța pachetelor

4.3 Analiza performanțelor și a rezilienței la rate limiting

Rezultatele testelor de viteză, latență și pierdere de pachete sunt prezentate în Figura 4.2. Graficele au fost generate folosind biblioteca *Matplotlib* [12]. Analiza metricilor arată impactul constrângerilor DNS asupra eficienței tunelului:

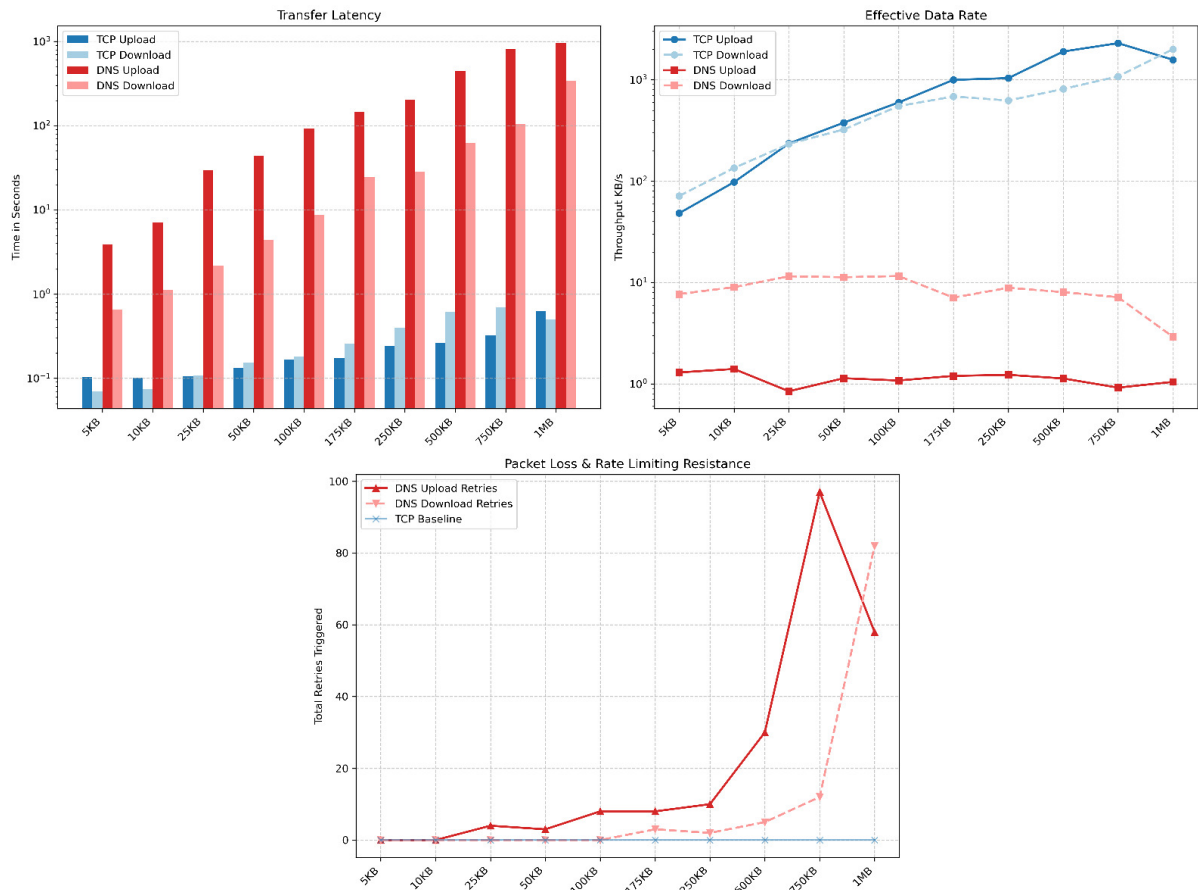


Figura 4.2: Latența, lățimea de bandă și reziliența la Rate Limiting a protocolului

- **Latența:** Asimetria dintre *Upload* și *Download* prin DNS este observabilă. Discrepanța validează ipoteza inițială: constrângerile de maxim 255 bytes per QNAME limitează viteza de *Upload*, pe când utilizarea extensiei EDNS în TXT permite un *Download* mai rapid. *Baseline*-ul TCP se remarcă prin terminarea transferurilor extrem de rapid bidirecțional.
- **Lățimea de bandă:** Graficul al doilea evidențiază un eveniment clasic protocoalelor cu *Sliding Window*: lățimea de bandă pentru TCP este proporțională dimensiunii fișierului, exemplificând *TCP Slow Start*. În contrast, *Throughput*-ul pentru tunelul DNS rămâne

aproape constant. Comportamentul acesta relevă eficiența metodei *Stop-and-Wait ARQ*, ce acționează ca un *traffic shaper*, fără a trimite pachetele în *burst*-uri.

- **Reziliența la Rate Limiting și Packet Loss:** Pentru fișierele de dimensiuni mici, de până la 250 KB, retransmisiile sunt minime, între 0 și 10 pachete. Începând cu pragul de 500 KB pierderile cresc alarmant, aproximativ 100 de retransmisii pentru *Upload* la fișierul de 750 KB. Această zonă marchează activarea filtrelor de *RRL*, ce încep să respingă cererile anormal de frecvente. Se observă că TCP-ul, protocol stabil ce nu suferă de *rate limiting*, nu prezintă vreo pierdere de pachete.

Deși multe pachete sunt pierdute din cauza mecanismelor de protecție ale rețelei, bucla de retransmisie a păstrat conexiunea activă. Integritatea tuturor fișierelor reconstruite a fost validată criptografic prin SHA-256, rezultat ce demonstrează robustețea sistemului de control al fluxului implementat și a protocolului la *rate limiting*.

5 Concluzii

5.1 Realizarea lucrării

Lucrarea a avut ca scop proiectarea și implementarea unui protocol de transport personalizat fiabil peste infrastructura DNS. Arhitectura a fost construită strict pe baza înregistrărilor de tip TXT și a interogărilor QNAME, utilizând protocolul UDP, ce ocupă portul 53. Prin dezvoltarea unui mecanism de controlare al fluxului de tip *Stop-and-Wait ARQ*, sistemul a rezolvat problema pierderii de pachete inerentă rețelelor instabile, fără stare a sesiunii, și a mitigat declanșarea excesivă a filtrelor de *Rate Limiting*. De asemenea, canalul de comunicație a fost securizat printr-un proces de *handshake* bazat pe ECDHE autentificat printr-un HMAC dependent de un PSK, urmat de criptarea autentificată AES-GCM a tuturor pachetelor în care sunt transmise date.

5.2 Rezultate și aplicații practice

Rezultatele obținute confirmă funcționalitatea protocolului în situații practice, demonstrând o reziliență ridicată în fața mecanismelor de apărare ale rețelei, chiar și în condiții de pierdere ma-

sivă de pachete. Deși viteza de transfer este foarte limitată, în special la transmiterea *upstream*, din cauza constrângerilor dimensionale ale protocolului DNS, acceptarea acestui compromis a fost necesară pentru a asigura o conexiune stabilă pe tot parcursul comunicării.

În practică, un astfel de sistem poate fi folosit ca *fallback* pentru a administra la distanță echipamente ce se află în medii foarte stricte cu comunicarea externă. În cazurile în care porturile folosite de obicei pentru management (precum SSH sau RDP) sunt blocate de *Firewall*-uri avansate, infrastructura DNS rămâne de cele mai multe ori unica modalitate de comunicație deschisă. Protocolul creat ar permite astfel executarea mentenanței de urgență sau recuperarea datelor din sistemele izolate.

5.3 Direcții de dezvoltare ulterioară

Deși obiectivele inițiale au fost îndeplinite de implementarea instrumentului, există mai multe căi prin care protocolul poate fi îmbunătățit ulterior:

- **Diminuarea entropiei informaționale:** Pentru micșorarea riscului de detecție din partea mecanismelor de tip NGFW, se poate adăuga un algoritm de (*padding*) bazat pe dicționare. Intercalarea datelor codificate în Base32 împreună cu șiruri de caractere ce aparțin limbajului natural ar reduce scorul entropic al etichetelor din domeniile generate. În plus, dacă s-ar adăuga un proces de dinamizare a dimensiunilor cererilor și răspunsurilor DNS, pericolul reprezentat de instrumentele pentru analiză euristică și statistică ar fi estompat.
- **Optimizarea controlului de flux:** Mecanismul strict de *Stop-and-Wait*, bazat pe o buclă cu maxim 6 încercări și un *timeout* de 3s pentru fiecare retransmisie, poate fi înlocuit cu o abordare mai flexibilă, fondată pe *Leaky Bucket*. Această opțiune ar permite modificarea dinamică a ratei de transmisie, perfecționând lățimea de bandă, mai ales la *download*, fără a trece pragurile de detecție ale filtrelor RRL.
- **Rotația domeniilor (*Domain Flipping*):** Obținerea mai multor domenii autoritative (inclusiv a name serverelor aferente) și alternarea lor în timpul sesiunilor de tunelare. Această funcționalitate poate contribui la prevenția blocării fluxului de transfer al protocolului prin adăugarea unui singur domeniu pe un *blacklist*.

Anexa 1 Parametrii curbei eliptice SECP256R1

Securitatea algoritmului ECDHE se bazează pe dificultatea calculării logaritmului discret pe o curbă eliptică peste $\mathbb{Z}_p$ definită de ecuația $y^2 = x^3 + ax + b \bmod p$, unde $4a^3 + 27b^2 \neq 0 \bmod p$. Constantele criptografice utilizate de protocol pentru curba SECP256R1 sunt extrase din NIST SP 800-186 [13] și sunt enumerate în Tabela A1.1.

[illegible]

Tabela A1.1: Specificatiile curbei eliptice NIST P-256

Anexa 2 Arhitectura operațională AES-GCM

Pentru criptarea autenticată protocolul folosește *AES* în modul *GCM*. Procesul este împărțit în fluxul de criptare/decriptare, pe baza funcției *GCTR*, și calcularea etichetei de autentificare prin funcția *GHASH*. Operațiile acestea sunt prezentate vizual în Figurile A2.1–A2.4, urmate de definițiile matematice, informațiile fiind preluate din *NIST Special Publication 800-38D* [5].

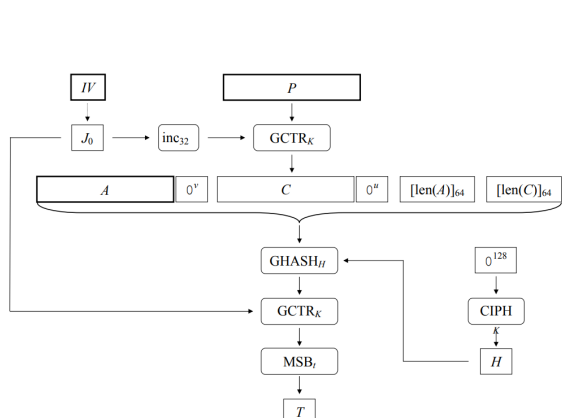


Figura A2.1: Criptare AES-GCM

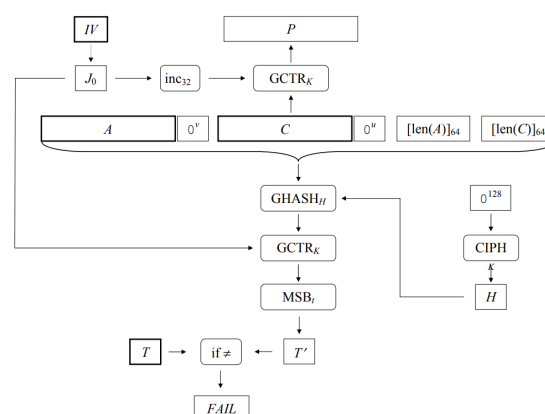


Figura A2.2: Decriptare AES-GCM

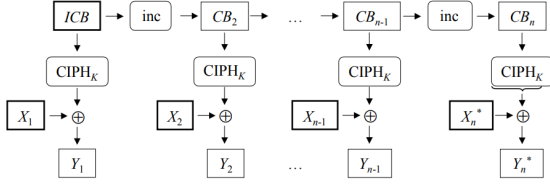


Figura A2.3: Operația GCTR

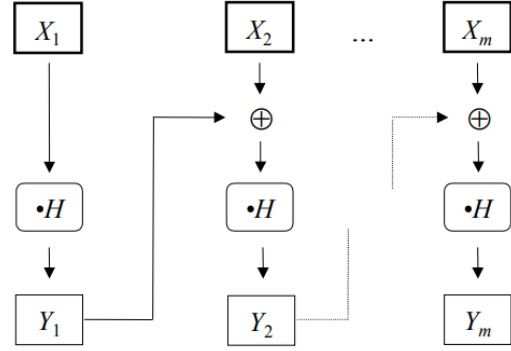


Figura A2.4: Generarea etichetei GHASH

Funcția GHASH Eticheta este calculată prin operații pe corpul finit Galois $GF(2^{128})$. Fie un șir de blocuri de 128 de biți $X_1, X_2, \dots, X_m$ și subcheia de hash $H = E_K(0^{128})$, funcția se definește iterativ:

$$Y_i = (Y_{i-1} \oplus X_i) \cdot H$$

unde $Y_0 = 0^{128}$, și rezultatul $Y_m = \text{GHASH}_H(X)$. Înmulțirea se realizează modulo polinomul ireductibil $P(x) = x^{128} + x^7 + x^2 + x + 1$.

Funcția GCTR Criptarea este realizată prin modul *CTR*. Pentru fiecare bloc de text în clar X_i și valoarea contorului CB_i , textul cifrat Y_i se calculează:

$$Y_i = X_i \oplus E_K(CB_i)$$

unde E_K este aplicarea algoritmului AES cu secretul K .

Criptarea autenticată Pentru a proteja atât datele adiționale (A), cât și textul cifrat (C), eticheta de autentificare finală T (de lungime t) este calculată prin aplicarea funcției GHASH asupra concatenării acestora, urmată de o mascare XOR cu valoarea inițială a contorului criptat (J_0):

$$T = \text{MSB}_t(\text{GHASH}_H(A \parallel 0^v \parallel C \parallel 0^u \parallel \text{len}(A)_{64} \parallel \text{len}(C)_{64}) \oplus E_K(J_0))$$

$$T' = \text{MSB}_t(\text{GHASH}_H(A \parallel 0^v \parallel C \parallel 0^u \parallel \text{len}(A)_{64} \parallel \text{len}(C)_{64}) \oplus E_K(J_0))$$
$$P = \begin{cases} \text{GCTR}_K(J_0, C), & \text{dacă } T = T' \\ \text{FAIL}, & \text{dacă } T \neq T' \end{cases}$$

Anexa 3 Analiza detaliată a pachetelor în Wireshark

```

Frame 2789: 314 bytes on wire (2512 bits), 14 bytes captured (2512 bits) on interface wlan0, id 6
Ethernet II, Src: Intel_E7:f6:f2 (18:51:07:57:f6:f2), Dst: ztc_3c:d:a3c (c8:b9:a4:c3:d:a3c)
Internet Protocol Version 4, Src: 192.168.1.4, Dst: 8.8.8.8
User Datagram Protocol, Src Port: 24856, Dst Port: 53
Domain Name System query
Transaction ID: 0x2f4c
Flags: 0xb0 Standard query
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 1
- Queries
    [uncated] NX(21224035PGLB1ZK1S66WMMHJQEOYOSVLSVEHYXHL5KDORSDTDEJBCTVP.WPDZA64GKJC2RBRHW?
Name [fruncated]: NXL(212403PSLGNBLZKS1566WMMHJSVEHYOVSLE.VXYHMLSKDORSOTDEJBCTVP.DBJEVC.JA64GKJCBWRHW?
[Name Length: 243]
Label Count: 9
Type: TXT (16) (Text strings)
Class: IN (0x0001)
- Additional records
-root> type opt
    [uncated] Q(21224035PGLB1ZK1S66WMMHJQEOYOSVLSVEHYXHL5KDORSDTDEJBCTVP.WPDZA64GKJC2RBRHW?)

```

Conform analizei din Capitolul 4, Figura A3.1 accentuează dimensiunea numelui de domeniu folosit în transferul de date. Similar, răspunsurile de la server prezintă structura conturată în Figura A3.2.

```

Frame 28892: 1267 bytes on wire (10136 bits), 1267 bytes captured (10136 bits) on interface wlp2s0,
Ethernet II, Src: zte_3c:da:38 (c0:94:ad:3c:da:38), Dst: Intel_57:f6:f2 (18:51:07:57:f6:f2)
Internet Protocol Version 4, Src: 8.8.8.8, Dst: 192.168.1.4
User Datagram Protocol, Src Port: 53, Dst Port: 32325
Domain Name System (response)
  Transaction ID: 0x7edf
  Flags: 0x8100 Standard query response, No error
  Questions: 1
  Answer RRs: 1
  Authority RRs: 0
  Additional RRs: 1
  Queries
    - WN61NOCC4XW74VVI243UUBFP1VWA35GNV1KZ7EZO.30.2930611582.server.ownprotocol.ninja: type TXT, cla
      Name: WN61NOCC4XW74VVI243UUBFP1VWA35GNV1KZ7EZO.30.2930611582.server.ownprotocol.ninja
      [Name Length: 79]
      [Label Count: 6]
      Type: TXT (16) (Text strings)
      Class: IN (0x0001)
  Answers
    - WN61NOCC4XW74VVI243UUBFP1VWA35GNV1KZ7EZO.30.2930611582.server.ownprotocol.ninja: type TXT, cla
      Name: WN61NOCC4XW74VVI243UUBFP1VWA35GNV1KZ7EZO.30.2930611582.server.ownprotocol.ninja
      Type: TXT (16) (Text strings)
      Class: IN (0x0001)
      Time to live: 0 (0 seconds)
      Data length: 1105
      TXT Length: 255
      TXT [truncated]: s3yguELl7f5m1KHQCQk8b1z0K6a15MdiIHsuHk1LVZDxyJH1Gb5MYZuUJ2N8ydw2ma56E5D
      TXT Length: 255
      TXT [truncated]: sN4gXrWCrwmpQaq3tn9AtztAA400lWihxqjdIIEcyZgdfT3Gauoc23hibf2BrEZgcPnHS7L+5v
      TXT Length: 255
      TXT [truncated]: dgbn0FKBquRip4ddi0uv08ppt5gN8SX4m3+99qPacdBbqAQRy18pD8by5f1IUrLUPuv68xJrBia
      TXT Length: 255
      TXT [truncated]: nQrJVmLj4l2k4IpTC34Qwey1T+B2au5mXfzdfGa0WYRPbOdTeVE9EjRuot5+WC0DR5Df2nhV5Tn
      TXT Length: 80
      TXT: jclEXjGkqjaNuy1oVipWuvPB6/lDmIrthHvrj]lWSgbvKBoS511hWUBZ/fweiXy04Dh5leTf+zKaETH0b
  Additional records
    <Root>: type OPT
    [Request In: 28891]

```

Figura A3.2: Detalierea răspunsului din downstream și a extensiei EDNS

Referințe

- [1] G. Farnham, *Detecting DNS Tunneling*, InfoSec Reading Room, SANS Institute, 2013, URL: <https://sansorg.egnyte.com/dl/wfX8phjMqgq7>, Accesat: 10 Iunie 2026.
- [2] Internet Systems Consortium (ISC), *Using Response Rate Limiting (RRL) in BIND 9*, Knowledge Base Article, Internet Systems Consortium, 2021, URL: <https://kb.isc.org/docs/aa-01000>, Accesat: 10 Iunie 2026.
- [3] E. Barker, L. Chen, A. Roginsky, A. Vassilev și R. Davis, *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*, NIST Special Publication 800-56A Rev. 3, National Institute of Standards și Technology (NIST), Apr. 2018, DOI: [10.6028/NIST.SP.800-56Ar3](https://doi.org/10.6028/NIST.SP.800-56Ar3), URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-56Ar3.pdf>, Accesat: 10 Iunie 2026.
- [4] H. Krawczyk și P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, Internet Engineering Task Force (IETF), Mai 2010, DOI: [10.17487/RFC5869](https://doi.org/10.17487/RFC5869), URL: <https://datatracker.ietf.org/doc/html/rfc5869>, Accesat: 10 Iunie 2026.
- [5] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, National Institute of Standards și Technology (NIST), Nov. 2007, DOI: [10.6028/NIST.SP.800-38D](https://doi.org/10.6028/NIST.SP.800-38D), URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>, Accesat: 10 Iunie 2026.
- [6] H. Krawczyk, M. Bellare și R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, Internet Engineering Task Force (IETF), Feb. 1997, DOI: [10.17487/RFC2104](https://doi.org/10.17487/RFC2104), URL: <https://datatracker.ietf.org/doc/html/rfc2104>, Accesat: 10 Iunie 2026.
- [7] E. Ekman și B. Andersson, *Iodine: IPv4 over DNS Tunnel*, 2023, URL: <https://code.kryo.se/iodine/>, Accesat: 10 Iunie 2026.

- [8] R. Bowes, *dnscat2: DNS-based Command and Control*, 2022, URL: <https://github.com/iagox86/dnscat2>, Accesat: 10 Iunie 2026.
- [9] B. Fox, *Sliver Command and Control Framework*, 2024, URL: <https://sliver.sh/docs>, Accesat: 10 Iunie 2026.
- [10] T. Perrin, *The Noise Protocol Framework*, Technical Specification, Noise Protocol, 2018, URL: <https://noiseprotocol.org/noise.html>, Accesat: 10 Iunie 2026.
- [11] Wireshark Foundation, *Wireshark Network Protocol Analyzer*, 2026, URL: <https://www.wireshark.org/>, Accesat: 10 Iunie 2026.
- [12] J. D. Hunter, „Matplotlib: A 2D Graphics Environment”, în *Computing in Science & Engineering* 9.3 (2007), pp. 90–95, DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55), URL: <https://matplotlib.org/>, Accesat: 10 Iunie 2026.
- [13] L. Chen, D. Moody, A. Regenscheid și K. Randall, *Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters*, NIST Special Publication 800-186, National Institute of Standards și Technology (NIST), Feb. 2023, DOI: [10.6028/NIST.SP.800-186](https://doi.org/10.6028/NIST.SP.800-186), URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-186.pdf>, Accesat: 10 Iunie 2026.